

ALU Logo Registry dApp Documentation

Demo video link:

<https://drive.google.com/file/d/17icWOlewDk3eft9A1ryqcoA2tcnnJReK/view?usp=sharing>

Abstract

The ALU Logo Registry dApp is a blockchain-based application that allows users to register, verify, and manage ownership information of digital assets such as the ALU official logo. The system uses smart contracts to create a tamper-resistant record of registered assets while providing a user-friendly interface for uploading files, generating hashes, verifying authenticity, and managing ALUT token ownership. The dApp solves the problem of unauthorized modification and misuse of digital assets by providing a transparent and verifiable ownership record.

Introduction

Smart contracts provide secure and immutable storage of asset ownership records, but they are not directly accessible or convenient for most users. Interacting with contracts directly requires technical knowledge of blockchain tools, wallet management, and transaction handling.

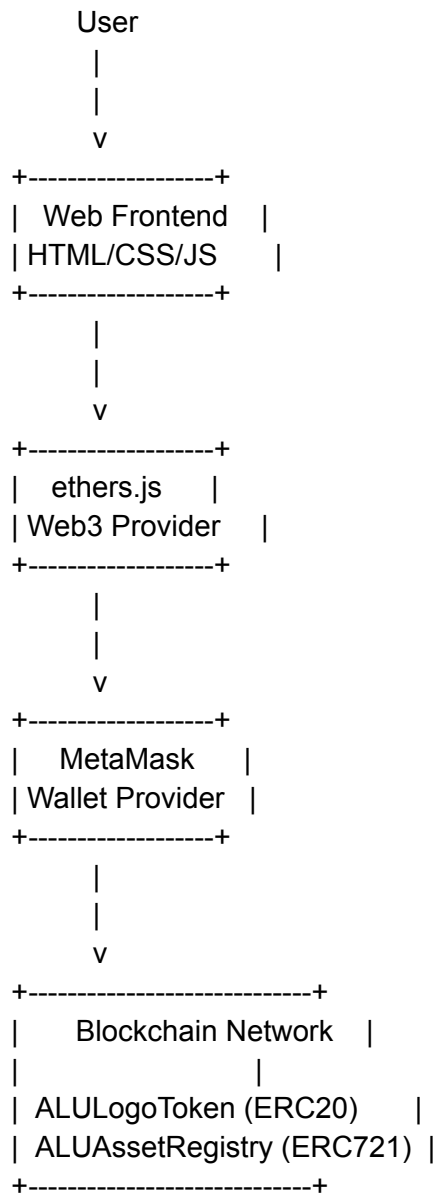
The ALU Logo Registry dApp fills this gap by providing a simple web interface that connects users to the blockchain. Users can upload a logo, generate a cryptographic hash, register the asset, and verify authenticity without needing to interact with contract functions manually. The frontend improves accessibility while preserving the security guarantees provided by blockchain technology.

Architecture Overview

The dApp follows a client-side Web3 architecture consisting of three main components:

1. **Frontend Application**
2. **Web3 Wallet**
3. **Smart Contracts**

Architecture Diagram



Communication Flow

1. The user opens the dApp in a browser.
2. The frontend detects the user's Web3 wallet.
3. MetaMask provides access to the blockchain through an Ethereum provider.
4. ethers.js creates contract instances using:
 - contract addresses
 - ABI files
 - wallet signer

5. Users can read blockchain data or submit transactions.
 6. Transactions are approved by the user through their wallet and recorded on-chain.
-

Feature Walkthrough

1. Home Page

Purpose: Provide wallet connection and display basic blockchain information.

Features:

- Connect wallet button
- Display connected wallet address
- Display current network
- Display ALUT token balance
- Display registered asset information

The page allows users to confirm that their wallet is correctly connected before interacting with blockchain features.

Screenshot:

(Insert home page screenshot here)

2. Logo Registration Page

Purpose: Allow users to register a new digital asset.

The registration process consists of three steps:

File Upload

Users select an image file from their device. The image is previewed in the browser before registration.

Hash Generation

The application generates a SHA-256 hash of the file contents. This hash acts as a unique digital fingerprint of the logo.

Asset Registration

The user enters:

- Asset name
- File type

The frontend then calls:

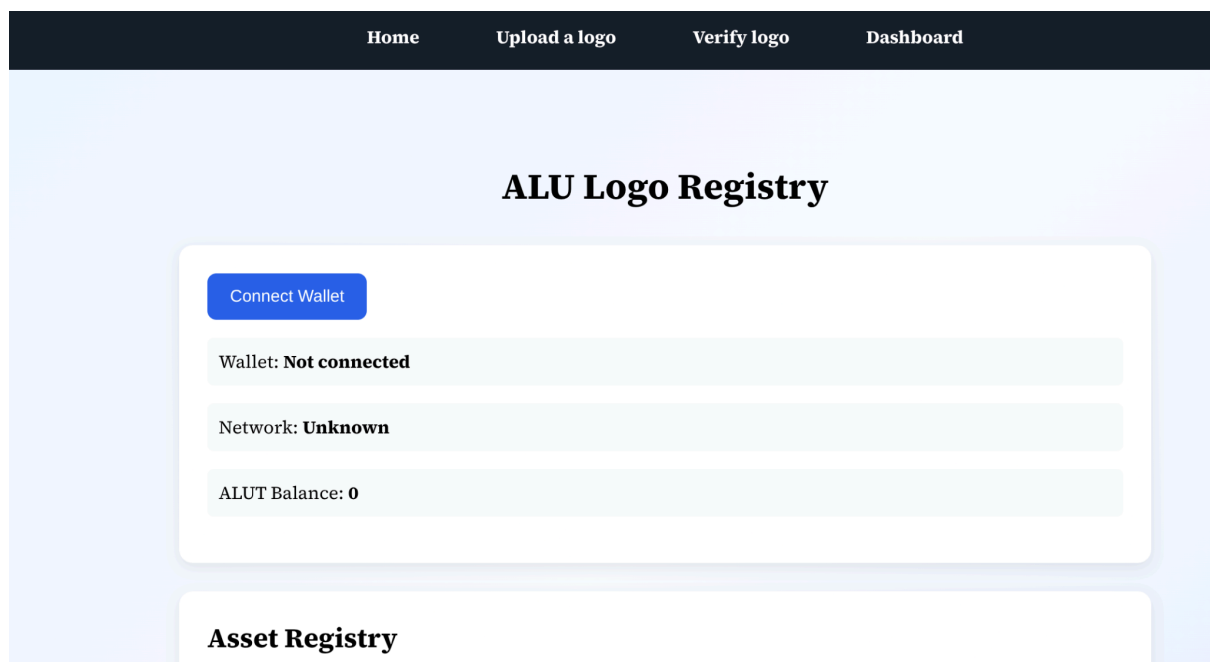
registerAsset()

on the [ALUAssetRegistry](#) contract.

After confirmation, the blockchain stores:

- Asset name
- File type
- SHA-256 hash
- Registering wallet
- Registration timestamp

Screenshot:



3. Logo Verification Page

Purpose: Allow anyone to verify whether a logo is authentic.

No wallet connection is required.

Users can verify using:

File Upload

The application:

1. Reads the uploaded file.
2. Generates its SHA-256 hash.
3. Sends the hash to the smart contract.
4. Compares it with the stored hash.

Hash Input

Users can also paste an existing SHA-256 hash directly.

The contract function:

`verifyLogoIntegrity()`

returns whether the supplied hash matches the registered asset.

Screenshot:

ALU Logo Registration

Wallet

Connected

Address: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Upload Logo

Choose File

 screenshot-2026-07-01_16-12-17.png

To go

Generate Hash

SHA-256 Hash

4. Token Dashboard

Purpose: Display ALUT ownership information and allow token distribution.

Features:

- Total ALUT supply display
- Connected wallet balance
- Ownership percentage
- Example token holders
- Owner-only token distribution form

The owner can distribute tokens using:

`distributeShares()`

The dashboard updates after successful blockchain confirmation.

Screenshot:

Wallet and Web3 Integration

The dApp uses MetaMask as the user's blockchain wallet and ethers.js as the communication layer between the frontend and smart contracts.

When a user clicks **Connect Wallet**, the frontend requests access to the wallet:

```
window.ethereum.request({
  method: "eth_requestAccounts"
});
```

MetaMask provides the user's account information.

ethers.js then creates a provider:

```
const provider =  
  new ethers.BrowserProvider(window.ethereum);
```

The provider allows the application to read blockchain data.

A signer is created:

```
const signer =  
  await provider.getSigner();
```

The signer allows the user to approve transactions such as registering assets or transferring tokens.

ethers.js handles:

- Contract calls
 - Transaction creation
 - Transaction signing
 - Reading blockchain state
-

Verification Page: In-Browser Hashing

The verification page performs hashing directly inside the user's browser using the Web Crypto API.

The process is:

User selects image

|
v

Browser reads file bytes

|
v

SHA-256 hashing

|
v

Convert hash to bytes32

|
v

Compare with blockchain record

The application does not require a backend server because the file never needs to leave the user's device.

Benefits:

- **Privacy:** The logo file is not uploaded to any server.
- **Security:** The same file always produces the same SHA-256 hash.
- **Decentralization:** Verification only requires access to the blockchain.
- **Efficiency:** The browser performs the computation locally.

The blockchain stores only the hash, not the actual image file. This reduces storage costs while still allowing anyone to verify authenticity.

Token Dashboard

The ALUT Token Dashboard provides a user-friendly interface for viewing token ownership information directly from the [ALULogoToken](#) smart contract. It displays the total token supply, the balance of the currently connected wallet, and the user's ownership percentage based on their share of the total supply.

The dashboard also displays example token holders and their ownership percentages, providing a clear overview of how ALUT tokens are distributed.

For token management, the dashboard includes a distribution feature that allows the contract owner to transfer ALUT tokens to other wallet addresses. The distribution function is protected using OpenZeppelin's [Ownable](#) access control mechanism.

The [distributeShares\(\)](#) function can only be called by the contract owner:

```
function distributeShares(
    address recipient,
    uint256 amount
) public onlyOwner {
    require(amount > 0, "Amount must be greater than zero.");

    _transfer(owner(), recipient, amount);
}
```

The [onlyOwner](#) modifier prevents unauthorized users from distributing tokens. If a non-owner wallet attempts to call this function, the transaction is rejected by the smart contract before any token transfer occurs.

This ensures that only the authorized account responsible for managing ALUT ownership can distribute shares.

Screenshot:

Connect Wallet

Address: **0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266**

Ownership Overview

Total Supply: **1000000.0** ALU

Your Balance: **758000.0** ALU

Your Ownership: **75 %**

Token Holders

Wallet	Ownership
0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266	75%

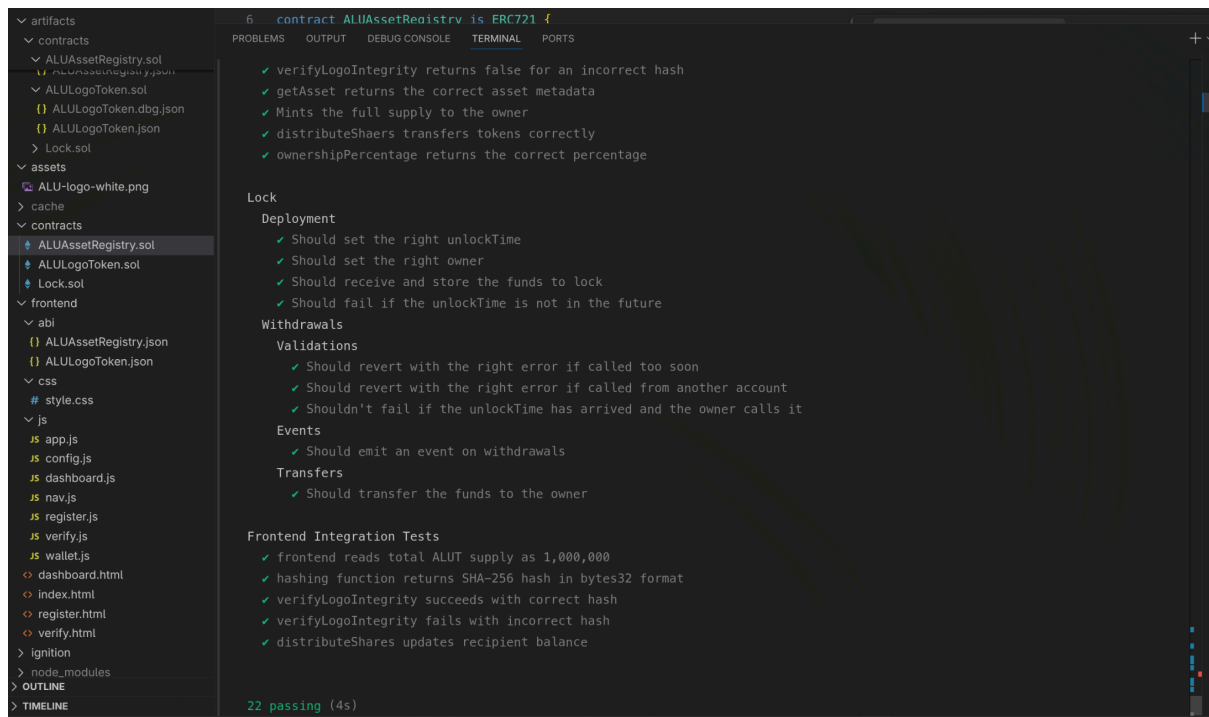
Test Results

The dApp was tested using Hardhat with a combination of smart contract unit tests and frontend integration tests.

The complete test suite contains thirteen tests:

- Eight original smart contract tests from Formative 1
- Five frontend integration tests covering:
 - Reading total ALUT supply
 - Browser SHA-256 hash generation
 - Successful logo verification
 - Failed logo verification
 - Token distribution balance updates

All thirteen tests passed successfully, confirming that the frontend correctly communicates with the deployed smart contracts and that the contract functions behave as expected.



Conclusion

Developing the ALU Logo Registry dApp provided practical experience in combining blockchain smart contracts with a user-facing Web3 application. I learned how decentralized applications use wallets, providers, and libraries such as ethers.js to allow users to interact with blockchain networks through a familiar web interface.

The project demonstrated how blockchain can be used beyond cryptocurrency by providing a transparent and tamper-resistant system for protecting digital assets. The use of SHA-256 hashing showed how files can be verified without storing the actual content on-chain, improving privacy and reducing storage requirements.

With more time, several improvements could be added:

- **Backend indexing:** Add a backend service or blockchain indexing solution to automatically track all token holders instead of using predefined example addresses.
- **Improved user authentication:** Add decentralized identity features to associate verified organizations with registered assets.
- **More asset support:** Extend the registry to support additional file types such as documents, certificates, and media files.
- **Better user experience:** Add transaction history, loading states, notifications, and improved mobile responsiveness.
- **IPFS integration:** Store actual asset files on decentralized storage while keeping the blockchain as the source of verification.

Overall, the project demonstrated how smart contracts and frontend technologies can work together to create practical decentralized applications that solve real-world problems.

